

```
In [ ]: import marimo as mo
```

Resumo do Trabalho Prático: Sudoku Genérico como CSP

Contexto & Visão Geral

O Sudoku é um exemplo clássico de um **Problema de Satisfação de Restrições (CSP)**. Em vez de codificar regras rígidas para linhas, colunas ou blocos de forma isolada, o objetivo deste trabalho é **abstrair a regra fundamental do Sudoku**: qualquer grupo de células pertence a um conjunto onde todos os valores têm de ser distintos (**AllDifferent**) e alguns valores podem estar previamente fixados (pistas).

Com esta abstração, linhas, colunas, blocos $n \times n$, pistas iniciais e até variantes (diagonais, hiper-sudoku, etc.) passam a ser tratados exatamente da mesma forma pelo solver.

Objetivos Principais

1. **Modelar** a abstração genérica de um grupo de células com a restrição "todos diferentes" (`box`).
 2. **Especializar** a classe genérica para construir blocos $n \times n$ (`cube`) e sequências retas horizontais/verticais (`path`).
 3. **Gerar pistas aleatórias** de forma paramétrica para criar tabuleiros de jogo.
 4. **Resolver** a grelha $n^2 \times n^2$ montando e satisfazendo o modelo CSP com o **OR-Tools (CP-SAT)**.
-

Requisitos Obrigatórios (R1 a R6)

- `box` — **Grupo Genérico (R1)**:
 - Guarda um mapeamento (linha, coluna) $\rightarrow$ valor ou None .
 - Método `add(i, j, val=None)` que adiciona células ao grupo e **rejeita** (levanta exceção) limites fora da grelha ou valores fora do intervalo $[1, n^2]$.
 - Método para exportar a representação como matriz $n^2 \times n^2$.
 - *Não sabe nada sobre Sudoku — apenas gere um grupo de células.*
- `cube` e `path` — **Especializações (R2 & R3)**:
 - **R2 (`cube`)**: Constrói o bloco $n \times n$ para os índices de bloco (i, j) .

- **R3 (path)**: Constrói o troço reto (linha ou coluna) entre duas coordenadas `inicio` e `fim` (funciona em qualquer sentido).
- **gerar_pistas — Pistas Aleatórias (R4)**:
 - Função que devolve um `box` com k células escolhidas aleatoriamente na grelha e fixadas a valores válidos $[1, n^2]$.
- **Modelo e Resolução CSP (R5 & R6)**:
 - **R5**: Modelo CSP com uma variável por célula $X_{i,j} \in [1, n^2]$. Aceita um número arbitrário de grupos (`box` , `cube` , `path` , pistas) e aplica a restrição `AllDifferent` e valores fixos.
 - **R6**: Junta todas as linhas, colunas, blocos $n \times n$ e o grupo de pistas para resolver a grelha.

Validação e Testes Automáticos

O notebook (ou módulo de testes) deve validar automaticamente:

1. **Integridade da Solução**: Confirmar que linhas, colunas e blocos contêm todos os valores de $1 \dots n^2$ sem repetições.
2. **Preservação de Pistas**: Garantir que os valores fixos gerados aleatoriamente se mantêm na solução final.
3. **Validação de Limites**: Confirmar que `add(...)` rejeita coordenadas ou valores inválidos.
4. **Generalização de Dimensão**: Executar o fluxo completo para $n = 3$ (9×9) e para outro valor como $n = 2$ (4×4) para provar que a solução não está *hardcoded*.

Extensões Opcionais (Bónus)

- **X-Sudoku (Diagonal)**: Adicionar as duas diagonais principais como grupos `box` com restrição `AllDifferent`.
- **Jigsaw / Sudoku Irregular**: Criar regiões de forma arbitrária como `box`.
- **Hyper-Sudoku / Windoku**: Adicionar 4 blocos sobrepostos suplementares.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontraste.
- **Sudoku 3D**: Estender a estrutura para grelhas $n^2 \times n^2 \times n^2$.

```
In [ ]: import random
        from ortools.sat.python import cp_model

        #1. Estrutura do box (R1) - dicionario + funcoes
        #Uma box é um dicionario:
        #{'name': str, 'n': int, 'cells': {(i,j):valor ou None}}
```

#None = celula livre; Int = celula fixa a esse valor
#Nao se sabe nada de linhas, colunas, blocos

```
def novo_box(n, name = "box", cells = None):
    #R1: cria um box vazio ou com cells iniciais se existirem
    box = {'name': name, 'n': n, 'cells': {}}
    if cells:
        for (i,j), val in cells.items():
            box_add(box, i, j, val)
    return box

def box_add(box, i, j, val=None):
    #R1: adiciona(i, j) ao box
    #Dá ValueError se as coordenadas ou valor forem invalidos
    n2 = box['n'] ** 2
    if not (0 <= i < n2 and 0 <= j < n2):
        raise ValueError(f"Coordenadas ({i},{j}) fora da grelha {n2}*{n2}")
    if val is not None and not (1<=val<=n2):
        raise ValueError(f"valor {val} fora do intervalo [1,{n2}]")
    box['cells'][(i,j)] = val

def box_matriz(box):
    #R1: Converte o box para uma matriz normal n2 x n2, preenche com 0 os espaços livres
    n2 = box['n'] ** 2
    m = [[0] * n2 for _ in range(n2)]
    for (i, j), v in box['cells'].items():
        m[i][j] = v if v is not None else 0
    return m
```

2. Formas concretas (R2, R3) e Pistas aleatorias (R4)

```
def generate_cube(n, i, j):
    #R2: gera um bloco n x n de índices de bloco (i, j), 0 <= i, j < n. Canto superior esquerdo = (i*n, j*n)
    if not (0 <= i < n and 0 <= j < n):
        raise ValueError(f"Índices de bloco ({i},{j}) fora dos limites([0,{n - 1}])")
    box = novo_box(n, f"bloco_{i}_{j}")
    for di in range(n):
        for dj in range(n):
            box_add(box, i * n + di, j * n + dj)
    return box

def generate_path(n, inicio, fim):
    #R3: cria uma linha ou coluna entre inicio e fim, inclusive, em qualquer sentido
    (i0, j0), (i1, j1) = inicio, fim
    if i0 != i1 and j0 != j1:
        raise ValueError("o caminho tem de estar na mesma linha ou coluna")
    box = novo_box(n, f"path_{inicio}_{fim}")
    #Linha horizontal
    if i0 == i1:
```

```

        passo = 1 if j1 >= j0 else -1
        for j in range(j0, j1 + passo, passo):
            box_add(box, i0, j)
    else:
        #coluna vertical
        passo = 1 if i1 >= i0 else -1
        for i in range(i0, i1 + passo, passo):
            box_add(box, i, j0)
    return box

def gerar_pistas(n, k=None):
    #R4: gera k pistas aleatorias(entre [1, n^2]) sem repetir posicoes nem
    valores;
    #k por omissão = n;
    #Os valores são distintos porque o grupo de pistas também é all-different
    n2 = n * n
    if k is None:
        k = n

    if not (0 <= k <= n2):
        raise ValueError(f"k={k} inválido: máximo {n2} pistas")

    box = novo_box(n, "pistas_aleatorias")
    todas = [(i, j) for i in range(n2) for j in range(n2)]
    #sorteia as posicoes e os numeros
    celulas = random.sample(todas, k)
    valores = random.sample(range(1, n2 + 1), k)

    for (i, j), v in zip(celulas, valores):
        box_add(box, i, j, v)
    return box

# 3. Modelo CSP (R5)

solucao = "solucao"
sem_solucao = "sem_solucao"
desconhecido = "desconhecido" #timeout: nao significa insolúvel

def build_and_solve(n, boxes, time_limit_s=10):
    #R5: uma variável por célula em [1, n^2]; para cada box impõe
    #all-different e fixa as células com valor
    #Devolve (grelha, estado): grelha é dict {(i,j): valor} ou None;
    #estado é solucao, sem_solucao ou desconhecido
    n2 = n * n

    # guardar os valores fixo e verifica se ha conflitos entre boxes/pistas
    fixos = {}
    for box in boxes:
        for pos, v in box['cells'].items():
            if v is None:
                continue
            if pos in fixos and fixos[pos] != v:
                return None, sem_solucao #conflito de pistas na mesma posicao

```

```

        fixos[pos] = v

model = cp_model.CpModel()
vars_map = {}
#criar as variaveis do solver para cada celula que aparece nos boxes
for box in boxes:
    for pos in box['cells']:
        i, j = pos
        if not (0 <= i < n2 and 0 <= j < n2):
            raise ValueError(f"Célula {pos} fora da grelha {n2}x{n2}.")
        if pos not in vars_map:
            vars_map[pos] = model.NewIntVar(1, n2, f"x_{i}_{j}")

#Aplicar os valores fixos ao modelo
for pos, v in fixos.items():
    model.Add(vars_map[pos] == v)

#Restricao, todos os elementos dentro de cada box tem que ser diferentes
for box in boxes:
    model.AddAllDifferent([vars_map[pos] for pos in box['cells']])

#configurar e executar o solver
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = time_limit_s
solver.parameters.num_search_workers = 8
status = solver.Solve(model)

if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    return {pos: solver.Value(var) for pos, var in vars_map.items()}, solucao
if status == cp_model.INFEASIBLE:
    return None, sem_solucao
return None, desconhecido

# 4. Montagem do sudoku (R6)

def montar_sudoku_completo(n, pistas_box):
    #Junta todas as regras:Linhas, colunas, blocos, pistas dadas
    n2 = n * n
    boxes = [pistas_box]

    #linhas e colunas
    for i in range(n2):
        boxes.append(generate_path(n, (i, 0), (i, n2 - 1)))# Linha i
        boxes.append(generate_path(n, (0, i), (n2 - 1, i))) # coluna i

    #blocos n*n
    for i in range(n):
        for j in range(n):
            boxes.append(generate_cube(n, i, j))

    return boxes

def resolver_sudoku(n, k=None, max_tentativas=50, time_limit_s=10):

```

```

    #Gera pistas aleatórias e resolve. Se o puzzle não tiver solução, tenta novas
    pistas (até max_tentativas)
    estado = sem_solucao
    for _ in range(max_tentativas):
        pistas = gerar_pistas(n, k)
        grelha, estado = build_and_solve(n, montar_sudoku_completo(n, pistas),
time_limit_s)
        if estado == solucao:
            return grelha, pistas, estado

    return None, None, estado

# 5. Validacao automatica

def validar_solucao(n, grelha, pistas_box):
    #Verifica se a solucao final cumpre todas as regras do Sudoku
    #Levanta AssertionError se a solução for inválida
    n2 = n * n
    esperado = set(range(1, n2 + 1))

    #Validar linhas e colunas
    for i in range(n2):
        assert {grelha[(i, j)] for j in range(n2)} == esperado, f"linha {i}
inválida"
        assert {grelha[(j, i)] for j in range(n2)} == esperado, f"coluna {i}
inválida"

    #validar blocos
    for bi in range(n):
        for bj in range(n):
            vals = {grelha[(bi * n + di, bj * n + dj)] for di in range(n) for dj
in range(n)}
            assert vals == esperado, f"bloco ({bi},{bj}) inválido"

    #Valida se manteve as pistas iniciais
    for pos, v in pistas_box['cells'].items():
        assert grelha[pos] == v, f"pista {pos}={v} não respeitada"

def testar_box_add():
    n = 3
    b = novo_box(n)
    box_add(b, 0, 0, 5)
    box_add(b, 1, 1)
    assert b['cells'] == {(0, 0): 5, (1, 1): None}
    assert box_matriz(b)[0][0] == 5 and box_matriz(b)[1][1] == 0

    #testa se rejeita inputs invalidos
    for args in [(9, 0, 1), (0, 9, 1), (-1, 0, 1), (0, -1, 1), (0, 0, 0), (0, 0,
10)]:
        try:
            box_add(b, *args)
        except ValueError:
            continue

```

```

        raise AssertionError(f"box_add{args} devia ter dado erro")
    print("box_add rejeita coordenadas e valores inválidos")

def testar_formas():
    n = 3
    assert sorted(generate_cube(n, 1, 2)['cells']) == sorted(
        (3 + di, 6 + dj) for di in range(3) for dj in range(3))
    assert list(generate_path(n, (2, 0), (2, 3))['cells']) == [(2, 0), (2, 1), (2,
2), (2, 3)]
    assert list(generate_path(n, (2, 3), (2, 0))['cells']) == [(2, 3), (2, 2), (2,
1), (2, 0)]
    assert list(generate_path(n, (0, 4), (2, 4))['cells']) == [(0, 4), (1, 4), (2,
4)]
    assert list(generate_path(n, (2, 4), (0, 4))['cells']) == [(2, 4), (1, 4), (0,
4)]
    print("cube e path corretos (nos dois sentidos)")

def testar_conflito_e_insolucao():
    n = 3
    a = novo_box(n, cells={(0, 0): 1})
    b = novo_box(n, cells={(0, 0): 2})
    _, estado = build_and_solve(n, [a, b])
    assert estado == sem_solucao
    # duas células na mesma linha com o mesmo valor
    c = novo_box(n, cells={(0, 0): 4, (0, 1): 4})
    d = generate_path(n, (0, 0), (0, 8))
    _, estado = build_and_solve(n, [c, d])
    assert estado == sem_solucao
    print("conflitos devolvem sem_solucao")

def testar_fluxo(n):
    grelha, pistas, estado = resolver_sudoku(n)
    assert estado == solucao, f"sem solução para n={n} ({estado})"
    validar_solucao(n, grelha, pistas)
    print(f"fluxo completo n={n} ({n * n}x{n * n}) validado")
    return grelha, pistas

def imprimir_grelha(n, grelha):
    #Imprime uma grelha dada como dict {(i,j): valor ou None}
    #Células sem valor aparecem como '.'
    n2 = n * n
    larg = len(str(n2))
    for i in range(n2):
        partes = []
        for j0 in range(0, n2, n):
            valores = []
            for j in range(j0, j0 + n):
                v = grelha.get((i, j))
                texto = "." if v is None else str(v)
                valores.append(texto.rjust(larg))
            partes.append(" ".join(valores))
        print(" | ".join(partes))

```

```

        if (i + 1) % n == 0 and i + 1 != n2:
            print("--".join("-" * (n * (larg + 1) - 1) for _ in range(n)))

if __name__ == "__main__":
    testar_box_add()
    testar_formas()
    testar_conflito_e_insolucao()
    testar_fluxo(2)
    testar_fluxo(3)

    # Demonstração, trocar valor do k para mudar o numero de pistas até n^2
    g, p, estado = resolver_sudoku(3, k=9)
    assert estado == solucao
    validar_solucao(3, g, p)

    print("\nPuzzle inicial (só as pistas):")
    imprimir_grelha(3, p['cells'])

    print("\nSolução:")
    imprimir_grelha(3, g)

```

box_add rejeita coordenadas e valores inválidos
 cube e path corretos (nos dois sentidos)
 conflitos devolvem sem_solucao
 fluxo completo n=2 (4x4) validado
 fluxo completo n=3 (9x9) validado

Puzzle inicial (só as pistas):

```

. . 6 | . 1 . | 5 . .
. . . | . . . | . . .
. . . | . . . | . . .
-----+-----+-----
. . 8 | . 9 . | . . .
. . . | 3 . . | . . .
. . . | . . . | . . .
-----+-----+-----
. . . | . . . | . . .
. 2 . | . . . | 4 . 7
. . . | . . . | . . .

```

Solução:

```

2 4 6 | 7 1 3 | 5 9 8
8 1 5 | 9 6 2 | 3 7 4
7 9 3 | 4 8 5 | 1 6 2
-----+-----+-----
1 6 8 | 2 9 4 | 7 3 5
4 7 9 | 3 5 1 | 8 2 6
5 3 2 | 6 7 8 | 9 4 1
-----+-----+-----
9 5 7 | 8 4 6 | 2 1 3
6 2 1 | 5 3 9 | 4 8 7
3 8 4 | 1 2 7 | 6 5 9

```

Explicação da Abordagem e Código

Como pensei a solução (`box`)

A ideia principal do trabalho foi não complicar. Em vez de criar regras separadas para linhas, colunas e blocos, fiz tudo a partir da mesma estrutura base: o `box`.

Um `box` é simplesmente uma lista/dicionário de posições da grelha que têm de ter números todos diferentes. O `box` em si não quer saber se é uma linha, uma coluna, um bloco $n \times n$ ou as pistas iniciais — para ele são apenas coordenadas. Esta abstração facilitou montagem do problema, porque o solver trata tudo exatamente da mesma maneira.

Construção das formas e das pistas

- `generate_cube` e `generate_path` : Criam os blocos $n \times n$ e as linhas/colunas retas. Reaproveitam a função `box_add` para garantir que nenhuma coordenada sai fora da grelha.
 - `gerar_pistas` : Sorteia k posições e k números. Como devolve um `box` normal com os valores fixados, entra no modelo CSP exatamente como se fosse outra regra qualquer da grelha.
-

Escolha do Solver (OR-Tools / CP-SAT)

Optei por usar a biblioteca **Google OR-Tools (CP-SAT)** por ter sido trabalhada durante as aulas e por ser bastante eficiente para problemas deste género(CSP).

- Cada célula da grelha passa a ser uma variável do solver com valores entre 1 e n^2 .
 - Para cada `box`, adiciona-se a restrição `AddAllDifferent`.
 - Para as pistas, diz-se ao solver que aquela variável tem de ser igual ao valor sorteado (`Add(var == val)`).
 - Antes de correr o solver, fiz também uma verificação simples em Python para apanhar logo conflitos diretos entre pistas nas mesmas posições.
-

Registo de Utilização de LLM

Durante a realização deste trabalho, utilizei ferramentas de IA (LLM) como apoio pontual para:

1. **Ajudar a estruturar os testes automatizados**: Criar casos de teste para validar se as exceções (`ValueError`) e a verificação de limites estavam a funcionar bem.
2. **Geração de exemplos de código**: Gerar exemplos de funções para usar na resolução do exercício de forma a orientar o melhor caminho a seguir.

3. **Revisão e limpeza de código:** Simplificar algumas mensagens de erro e garantir que os comentários do código ficavam claros e legíveis.
4. **Formatação no Marimo:** Organizar a apresentação final do notebook e a disposição das células de texto em Markdown.
5. **Validação automática (Criação de Testes e Validação de Funções):**
 - **Elaboração de Casos de Teste:** Apoio na definição da lógica das funções de teste (`testar_box_add` , `testar_formas` , `testar_conflito_e_insolucao`) para garantir que o código rejeita entradas inválidas (coordenadas fora dos limites e valores fora do intervalo $[1, n^2]$) e lança as exceções `ValueError` adequadas.
 - **Validação de Soluções:** Construção da função `validar_solucao` para confirmar automaticamente se as soluções geradas mantêm as pistas iniciais intactas e cumprem a regra `AllDifferent` em todas as linhas, colunas, blocos e diagonais/variantes
 - **Demonstração Visual de Prova:** Auxílio na criação das funções de demonstração visual (`demonstrar_diagonais_x_sudoku` , `demonstrar_blocos_hyper_sudoku` , `demonstrar_regioes_jigsaw`), permitindo extrair e exibir os valores de cada grupo específico para provar a correção do solver

Link do Chat improvisado usado na resolução

Durante a resolução foi usado outro chat que foi perdido após emprestar a conta a terceiros.

<https://share.gemini.google/90VgxBJponwj>

X-SUDOKU (DIAGONAL)

```
In [ ]: # EXTRA 1: X-SUDOKU (DIAGONAL) COM DEMONSTRAÇÃO VISUAL
```

```
def generate_diagonais(n):  
    #Gera as duas diagonais principais como grupos box  
    n2 = n * n  
  
    diag1 = novo_box(n, "diag_principal")  
    for i in range(n2):  
        box_add(diag1, i, i)  
  
    diag2 = novo_box(n, "diag_secundaria")  
    for i in range(n2):  
        box_add(diag2, i, n2 - 1 - i)  
  
    return [diag1, diag2]  
  
def montar_x_sudoku(n, pistas_box):
```

```

#Linhas + colunas + blocos + pistas + 2 diagonais
boxes = montar_sudoku_completo(n, pistas_box)
boxes.extend(generate_diagonais(n))
return boxes

def resolver_x_sudoku(n, k=None, max_tentativas=50, time_limit_s=10):
    #Gera pistas e resolve o X-Sudoku.
    estado = sem_solucao
    for _ in range(max_tentativas):
        pistas = gerar_pistas(n, k)
        boxes = montar_x_sudoku(n, pistas)
        grelha, estado = build_and_solve(n, boxes, time_limit_s)
        if estado == solucao:
            return grelha, pistas, estado
    return None, None, estado

def demonstrar_diagonais_x_sudoku(n, grelha):
    #Extrai e imprime as duas diagonais para provar a restrição X-Sudoku
    n2 = n * n

    # Extrair valores
    diag_prin = [grelha[(i, i)] for i in range(n2)]
    diag_sec = [grelha[(i, n2 - 1 - i)] for i in range(n2)]

    print("\nProva de Validade das Diagonais")
    print(f"Diagonal Principal (\) : {diag_prin}")
    print(f"Diagonal Secundária (/): {diag_sec}")

    esperado = set(range(1, n2 + 1))
    assert set(diag_prin) == esperado, "Erro: Diagonal principal tem repetições!"
    assert set(diag_sec) == esperado, "Erro: Diagonal secundária tem repetições!"
    print("Ambas as diagonais contêm todos os números de 1 a", n2, "sem
repetições!")

# --- TESTE INTEGRADO X-SUDOKU ---
print("\nTESTE: X-Sudoku (Diagonal) 9x9 (n=3)")
g_x, p_x, est_x = resolver_x_sudoku(3, k=9)
assert est_x == solucao, "Erro ao resolver X-Sudoku"

print("\nPuzzle inicial (só as pistas):")
imprimir_grelha(3, p_x['cells'])

print("\nSolução (respeita linhas, colunas, blocos E as 2 diagonais):")
imprimir_grelha(3, g_x)

# Demonstração explícita da diferença nas diagonais
demonstrar_diagonais_x_sudoku(3, g_x)

```

1. TESTE: X-Sudoku (Diagonal) 9x9 (n=3)

Puzzle inicial (só as pistas):

```
. . . | 7 . . | . . .  
. 2 . | . . 5 | . . .  
. 4 . | . . . | . . .  
-----+-----+-----  
. . . | . . . | . . .  
. . . | . . . | . . .  
. . . | . . . | . . .  
-----+-----+-----  
3 . 6 | . 1 . | . . .  
. . . | . 8 . | . . .  
9 . . | . . . | . . .
```

Solução (respeita linhas, colunas, blocos E as 2 diagonais):

```
8 9 5 | 7 3 1 | 4 6 2  
6 2 3 | 9 4 5 | 7 1 8  
1 4 7 | 2 6 8 | 3 9 5  
-----+-----+-----  
5 1 8 | 3 9 4 | 2 7 6  
4 6 2 | 1 5 7 | 8 3 9  
7 3 9 | 8 2 6 | 1 5 4  
-----+-----+-----  
3 5 6 | 4 1 2 | 9 8 7  
2 7 1 | 5 8 9 | 6 4 3  
9 8 4 | 6 7 3 | 5 2 1
```

PROVA DE VALIDADE DAS DIAGONAIS

Diagonal Principal (\) : [8, 2, 7, 3, 5, 6, 9, 4, 1]

Diagonal Secundária (/): [2, 1, 3, 4, 5, 8, 6, 7, 9]

Ambas as diagonais contêm todos os números de 1 a 9 sem repetições!

Cria dois grupos box extra (diag_principal e diag_secundaria) com as coordenadas (i, i) e $(i, n^2 - 1 - i)$. A função montar_x_sudoku junta estes dois boxes à lista do Sudoku completo, aplicando a restrição AllDifferent nas diagonais sem alterar o solver.

Hyper-Sudoku (Windoku)

```
In [ ]: # EXTRA 2: HYPER-SUDOKU (WINDOKU) COM DEMONSTRAÇÃO VISUAL
```

```
def generate_hyper_blocks(n):  
    #Gera os 4 blocos interiores do Hyper-Sudoku  
    boxes = []  
    offsets = [1, n + 2] if n >= 2 else [1, n + 1]  
  
    count = 0  
    for bi in offsets:  
        for bj in offsets:  
            box = novo_box(n, f"hyper_bloco_{count}")  
            for di in range(n):
```

```

        for dj in range(n):
            box_add(box, bi + di, bj + dj)
        boxes.append(box)
        count += 1

    return boxes

def montar_hyper_sudoku(n, pistas_box):
    #Linhas + colunas + blocos + pistas + 4 blocos hyper
    boxes = montar_sudoku_completo(n, pistas_box)
    boxes.extend(generate_hyper_blocks(n))
    return boxes

def resolver_hyper_sudoku(n=3, k=9, max_tentativas=50, time_limit_s=10):
    #Gera pistas e resolve o Hyper-Sudoku
    estado = sem_solucao
    for _ in range(max_tentativas):
        pistas = gerar_pistas(n, k)
        boxes = montar_hyper_sudoku(n, pistas)
        grelha, estado = build_and_solve(n, boxes, time_limit_s)
        if estado == solucao:
            return grelha, pistas, estado
    return None, None, estado

def demonstrar_blocos_hyper_sudoku(n, grelha):
    #Extrai e imprime as 4 janelas do Hyper-Sudoku para provar a restrição
    n2 = n * n
    esperado = set(range(1, n2 + 1))
    offsets = [1, n + 2] if n == 3 else [1, n + 1]

    print("\nProva de Validade dos 4 Blocos Hyper (WINDOKU)")
    count = 1
    for bi in offsets:
        for bj in offsets:
            valores = [grelha[(bi + di, bj + dj)] for di in range(n) for dj in
range(n)]
            print(f"Janela Hyper #{count} (canto top ({bi},{bj})): {valores}")
            assert set(valores) == esperado, f"Erro na janela Hyper #{count}"
            count += 1

    print("As 4 janelas interiores contêm todos os números sem repetições!")

#TESTE INTEGRADO HYPER-SUDOKU
print("\nTESTE: Hyper-Sudoku / Windoku 9x9 (n=3)")
g_h, p_h, est_h = resolver_hyper_sudoku(3, k=9)
assert est_h == solucao, "Erro ao resolver Hyper-Sudoku"

print("\nPuzzle inicial (só as pistas):")
imprimir_grelha(3, p_h['cells'])

print("\nSolução (respeita também os 4 blocos interiores sobrepostos):")
imprimir_grelha(3, g_h)

```

```
# Demonstração explícita da diferença nos 4 blocos interiores
demonstrar_blocos_hyper_sudoku(3, g_h)
```

TESTE: Hyper-Sudoku / Windoku 9x9 (n=3)

Puzzle inicial (só as pistas):

```
. . . | 2 . . | . 1 .
. . . | . . . | . . 7
. . . | 5 . . | . . .
-----+-----+-----
. 3 . | . . . | . . .
. . 9 | 4 . . | . . .
. . . | . . . | . . .
-----+-----+-----
. . . | . . . | . . .
. . 8 | . . . | . . .
. . . | . 6 . | . . .
```

Solução (respeita também os 4 blocos interiores sobrepostos):

```
4 5 6 | 2 7 8 | 3 1 9
3 9 2 | 6 1 4 | 5 8 7
8 1 7 | 5 9 3 | 6 2 4
-----+-----+-----
6 3 4 | 8 2 1 | 9 7 5
1 8 9 | 4 5 7 | 2 6 3
7 2 5 | 9 3 6 | 8 4 1
-----+-----+-----
9 4 3 | 1 8 2 | 7 5 6
5 6 8 | 7 4 9 | 1 3 2
2 7 1 | 3 6 5 | 4 9 8
```

Prova de Validade dos 4 Blocos Hyper (WINDOKU)

Janela Hyper #1 (canto top (1,1)): [9, 2, 6, 1, 7, 5, 3, 4, 8]

Janela Hyper #2 (canto top (1,5)): [4, 5, 8, 3, 6, 2, 1, 9, 7]

Janela Hyper #3 (canto top (5,1)): [2, 5, 9, 4, 3, 1, 6, 8, 7]

Janela Hyper #4 (canto top (5,5)): [6, 8, 4, 2, 7, 5, 9, 1, 3]

As 4 janelas interiores contêm todos os números sem repetições!

Cria 4 grupos box adicionais com as coordenadas dos cantos internos da grelha (com um offset de 1 célula em relação às bordas). A função `montar_hyper_sudoku` junta estes 4 blocos sobrepostos à lista de restrições para serem resolvidos em conjunto com os blocos normais.

Jigsaw (Sudoku Irregular)

```
In [ ]: def generate_jigsaw_region(n, nome, lista_coordenadas):
          #Cria um box a partir de uma lista de coordenadas arbitrarias
          n2 = n * n
          if len(lista_coordenadas) != n2:
              raise ValueError(f"Região tem de ter exatamente {n2} células")
          box = novo_box(n, nome)
```

```

for i, j in lista_coordenadas:
    box_add(box, i, j)
return box

def montar_jigsaw_sudoku(n, regioes_jigsaw, pistas_box):
    #Linhas + colunas + regiões irregulares + pistas
    n2 = n * n
    boxes = [pistas_box]
    for i in range(n2):
        boxes.append(generate_path(n, (i, 0), (i, n2 - 1)))
        boxes.append(generate_path(n, (0, i), (n2 - 1, i)))
    boxes.extend(regioes_jigsaw)
    return boxes

def criar_regioes_jigsaw_exemplo_4x4(n=2):
    #Cria 4 regiões de exemplo para grelha 4x4 (n=2)
    # Regiões irregulares em formato de "L" / "T" para 4x4:
    r0 = [(0, 0), (0, 1), (0, 2), (1, 0)] # Formato L
    r1 = [(0, 3), (1, 1), (1, 2), (1, 3)] # Formato L invertido
    r2 = [(2, 0), (2, 1), (2, 2), (3, 0)] # Formato L
    r3 = [(3, 1), (3, 2), (3, 3), (2, 3)] # Formato L invertido
    return [
        generate_jigsaw_region(n, "jigsaw_0", r0),
        generate_jigsaw_region(n, "jigsaw_1", r1),
        generate_jigsaw_region(n, "jigsaw_2", r2),
        generate_jigsaw_region(n, "jigsaw_3", r3)
    ]

def resolver_jigsaw_sudoku(n=2, regioes=None, k=4, max_tentativas=50,
time_limit_s=10):
    #Gera pistas e resolve o Jigsaw Sudoku
    if regioes is None:
        regioes = criar_regioes_jigsaw_exemplo_4x4(n)
    estado = sem_solucao
    for _ in range(max_tentativas):
        pistas = gerar_pistas(n, k)
        boxes = montar_jigsaw_sudoku(n, regioes, pistas)
        grelha, estado = build_and_solve(n, boxes, time_limit_s)
        if estado == solucao:
            return grelha, pistas, estado
    return None, None, estado

def demonstrar_regioes_jigsaw(n, grelha, regioes):
    #Extrai e imprime o conteúdo de cada região irregular (Jigsaw)
    n2 = n * n
    esperado = set(range(1, n2 + 1))

    print("\nProva de Validade das regiões JIGSAW (irregulares)")
    for reg in regioes:
        nome = reg['name']
        coords = list(reg['cells'].keys())
        valores = [grelha[pos] for pos in coords]

```

```

        print(f"Região '{nome}' {coords}: {valores}")
        assert set(valores) == esperado, f"Erro na região {nome}"

    print("Todas as regiões de formato customizado têm os números sem repetições")

# --- TESTE INTEGRADO JIGSAW ---
print("\nTESTE: Jigsaw / Sudoku Irregular 4x4 (n=2)")
regioes_test = criar_regioes_jigsaw_exemplo_4x4(2)
g_j, p_j, est_j = resolver_jigsaw_sudoku(n=2, regioes=regioes_test, k=4)
assert est_j == solucao, "Erro ao resolver Jigsaw Sudoku"

print("\nPuzzle inicial (só as pistas):")
imprimir_grelha(2, p_j['cells'])

print("\nSolução (respeita as regiões de formato customizado):")
imprimir_grelha(2, g_j)

# Demonstração explícita da diferença nas regiões irregulares
demonstrar_regioes_jigsaw(2, g_j, regioes_test)

```

TESTE: Jigsaw / Sudoku Irregular 4x4 (n=2)

Puzzle inicial (só as pistas):

```

. 2 | . .
. 3 | . .
----+----
1 4 | . .
. . | . .

```

Solução (respeita as regiões de formato customizado):

```

3 2 | 1 4
4 3 | 2 1
----+----
1 4 | 3 2
2 1 | 4 3

```

PROVA DE VALIDADE DAS REGIÕES JIGSAW (IRREGULARES)

Região 'jigsaw_0' [(0, 0), (0, 1), (0, 2), (1, 0)]: [3, 2, 1, 4]

Região 'jigsaw_1' [(0, 3), (1, 1), (1, 2), (1, 3)]: [4, 3, 2, 1]

Região 'jigsaw_2' [(2, 0), (2, 1), (2, 2), (3, 0)]: [1, 4, 3, 2]

Região 'jigsaw_3' [(3, 1), (3, 2), (3, 3), (2, 3)]: [1, 4, 3, 2]

Todas as regiões de formato customizado têm os números sem repetições

A função `generate_jigsaw_region` recebe uma lista arbitrária de n^2 coordenadas (i, j) e guarda-as num box. Na função `montar_jigsaw_sudoku`, mantêm-se as linhas e colunas normais, mas substituem-se os blocos quadrados cube por estas regiões de formato livre.

Escala

```
In [ ]: import time

def testar_desempenho_e_escalas():
    print("Teste de Escala e Eficiência")
    for n in [2, 3, 6, 7]: # Testa 4x4, 9x9 e 36x36
        inicio = time.time()
        grelha, pistas, estado = resolver_sudoku(n, k=n*2, time_limit_s=15)
        duracao = time.time() - inicio

        if estado == solucao:
            validar_solucao(n, grelha, pistas)
            print(f"Tamanho n={n} ({n*n}x{n*n}): Resolvido em {duracao:.3f} segundos.")
        else:
            print(f"Tamanho n={n}: Estado '{estado}' após {duracao:.3f} segundos.")

testar_desempenho_e_escalas()
```

```
Teste de Escala e Eficiência
Tamanho n=2 (4x4): Resolvido em 0.016 segundos.
Tamanho n=3 (9x9): Resolvido em 0.064 segundos.
Tamanho n=6 (36x36): Resolvido em 9.900 segundos.
Tamanho n=7: Estado 'desconhecido' após 751.084 segundos.
```

Desempenho e Testes

- **Grelhas normais** ($n = 2$ e $n = 3$): O programa resolve tudo quase instantaneamente, como pode ser visto ao correr o programa.
- **Escala** ($n = 6$): À medida que o tabuleiro cresce para 36×36 , o número de combinações aumenta bastante, mas o CP-SAT continua a conseguir resolver o problema, mais lento mas consegue sem ser preciso mudar rigorosamente nada na lógica do código.
- **Escala** ($n \geq 7$): Apartir deste ponto, pelo menos na minha máquina, começou a não conseguir encontrar nova solução mesmo após 750 segundos.